How to Write a Final Year Project Report

A Thesis Guide

Darkhan Mentor

2026

Abstract

The final year project is the largest piece of independent work most undergraduate students undertake. It asks a student to choose a problem, design a solution, build it, test it, and explain what was learned — all within a single academic year, and all in a written report that examiners can judge. This guide follows that journey from start to finish: choosing a topic, writing a proposal, planning the work, studying the background, designing and building the solution, testing it, presenting the results, and writing them up clearly.

The guide is written for students in computing, engineering, business and the applied sciences, where the final year project usually produces something that works: a piece of software, a device, a system, a study or a plan. It explains the standard structure of the project report, chapter by chapter, with worked examples drawn from fictional projects so that students can see the method without confusing illustration with real data. It also covers the parts of the project that are not writing at all — time management, working with a supervisor, preparing a demonstration and facing the viva — because a project that is built well but reported badly, or reported well but never finished, earns neither mark.

The emphasis throughout is practical and honest. A good final year project is not the most ambitious one; it is the one that is finished, tested and explained. A student who follows this guide should be able to choose a project of the right size, plan it in weeks rather than wishes, build it in stages that can be shown to a supervisor, and write a report that states plainly what was built, how it was tested, what the results were, and what should happen next.

Keywords: final year project, project report, proposal, design, implementation, testing, evaluation, viva

Table of Contents

Abstract	2
1. What the Final Year Project Is	3
2. Choosing a Project Topic	5
3. Writing the Project Proposal	6
4. Planning and Timeline: Work Breakdown and Gantt-Style Planning	7
5. Literature and Background Study	9
6. Design and Methodology	10
7. Implementation and Development	11
8. Testing and Evaluation	12
9. Results	14
10. Discussion	15
11. Conclusion and Future Work	16
12. Report Structure and Formatting	17
13. The Presentation, Demonstration and Viva	19
14. Working with Your Supervisor	20
15. Common Mistakes and a Checklist	21
16. Conclusion	22
Suggested Reading	22

How to use this guide: read Chapters 1 to 4 before you choose a topic or write a proposal. Read Chapter 5 while you study the background, and Chapters 6 to 8 while you design, build and test. Use Chapters 9 to 12 while you write the report, Chapter 13 before the presentation and viva, and Chapter 15 as a final check before submission.

Chapter 1: What the Final Year Project Is

The final year project is a long piece of independent work, usually completed in the last year of a degree, in which a student identifies a real problem, designs a solution, builds or tests it, and writes a report explaining the whole process. It is different from every other assignment in the degree. In other assignments, the question is set by the teacher and the answer is known. In the final year project, the student chooses the question, and the answer is not known in advance. That independence is the point: the project shows that the student can work like a professional — defining a problem, managing time, making decisions and communicating results — and not only pass examinations.

Most final year projects take one of four forms. A development project builds something: an application, a website, a device or a system. A research project investigates a question, usually by collecting and analysing data. An investigation

project studies an existing system and proposes improvements. A design project produces a detailed design, sometimes with a working prototype. The form differs, but the structure of the report is similar in all four: a problem, a background study, a method, results, and a discussion of what the results mean.

1.1 How the project is assessed

Assessment usually has three parts: the written report, the product or artefact itself, and the presentation or viva. The report normally carries the largest share of the marks, because it is the lasting record of what was done and why. The product is judged on whether it works, whether it meets its requirements, and whether it was built soundly. The presentation and viva test whether the student understands the work and can defend it. A student who builds a brilliant system but cannot explain it, or writes a beautiful report about a system that does not work, will lose marks in both places. The project must be balanced: the building, the writing and the explaining all matter.

1.2 What examiners look for

Examiners look for five things. First, a clear problem and a clear aim: they should be able to state, after reading the first chapter, what the project set out to do. Second, evidence of independent work: the student chose, planned and carried out the project, with the supervisor advising rather than directing. Third, sound method: the design and the testing follow from the problem, and the choices are justified. Fourth, honest results: the report shows what worked and what did not, with evidence. Fifth, clear writing: the report is organised, the figures are labelled, and the language is plain. Originality is welcome, but it is not required at undergraduate level. A solid, finished, well-tested project on a familiar problem is worth more than an ambitious idea that was never completed.

Worked Example: A project aim, well and badly stated

Weak: 'This project is about mobile apps for students.' It names no problem and no outcome. Strong: 'This project aims to design and build a mobile application that helps first-year students find and book study rooms on campus, and to test whether it reduces the time students spend searching for a free room.' The strong version names the product, the users, the problem and the test. An examiner reading it knows exactly what success would look like.

[ ] I know which form of project my department accepts.

[ ] I know how the marks are divided between report, product and presentation.

[ ] I can state the aim of my project in one sentence.

[ ] I know the submission deadline and the rules for extensions.

Chapter 2: Choosing a Project Topic

Choosing a topic is the first real decision of the project, and it is the one that most affects everything after it. A good topic is interesting to the student, small enough to finish in the time available, and clear enough that success can be judged. Students usually fail here in two opposite ways: they choose a topic that is far too large, or they choose one that is so small or so vague that there is nothing to write about. The cure is the same in both cases: test the topic against the time, the skills and the resources you actually have, before you commit to it.

2.1 Where good topics come from

Good topics come from real problems, and real problems are everywhere. A problem you have experienced yourself — a slow process at your university, a task your family business does by hand, a difficulty faced by a local shop or clinic — is a strong starting point, because you understand the users and you care about the answer. Problems suggested by a supervisor, an employer or a local organisation are also good, because they come with a real user who can test the result.

2.2 Testing a topic before committing

Before you commit, test the topic against five questions. Is the problem real and specific? Can the work be finished in the time available, with a margin for things going wrong? Do you have, or can you learn in time, the skills and tools it needs? Can you get the data, the users or the materials it requires? And can success be measured — is there a test that would show whether the solution works? If the answer to any of these is no, narrow the topic until the answers become yes. Narrowing is not a defeat; it is the most important skill in project work. A project that does one thing well beats a project that promises five things and delivers none.

2.3 The topic statement

Write the topic as a short statement: the problem in one sentence, the proposed solution in one sentence, and the test of success in one sentence. Show this statement to your supervisor and to one other person, and ask them to explain it back to you. If they cannot, the topic is not yet clear enough. Keep the statement and return to it whenever the project threatens to grow: every new feature, every extra chapter, every side investigation should be tested against it. If a proposed addition does not serve the statement, it does not belong in the project.

Test question	A good sign	A warning sign
Is the problem specific?	One problem, one group of users	A whole industry, a vague goal

Can it be finished in time?	A working version in two thirds of the time	Needs skills you cannot learn in time

Can you get what you need?	Data, users and tools already available	Depends on someone else's permission

Can success be measured?	A clear test with a number or a comparison	Success means 'it looks nice'

[ ] The topic comes from a real problem I understand.

[ ] The problem, the solution and the test of success are each one sentence.

[ ] The work fits the time available with a margin for setbacks.

[ ] I have, or can learn in time, the skills and tools needed.

[ ] My supervisor has agreed the topic statement.

Chapter 3: Writing the Project Proposal

The proposal is a short document — usually five to ten pages — that asks the department to approve the project. It is also the project's first plan, and it is written before the real work begins. A good proposal does four things: it states the problem, it shows that the problem is worth solving, it explains how the student will solve it, and it gives a realistic plan for doing so. Supervisors and approval panels read many proposals, and they approve the ones that are clear, honest and feasible. They reject the ones that are vague, over-ambitious, or copied from elsewhere.

3.1 What the proposal must contain

A standard proposal has six parts. The introduction states the problem, the background and the aim. The objectives list the concrete steps — usually four to six — that will achieve the aim. The background section summarises what already exists, including similar systems or studies, and says what is missing. The methodology describes how the project will be carried out: the design approach, the tools, the data and the testing. The plan gives the timeline, usually as a table or a simple chart. The expected outcomes state what will be delivered: the product, the report, and the measure of success. Keep each part short; the proposal is a promise, not the report itself.

3.2 Writing objectives that can be checked

Objectives are the promises the project will be judged against, so they must be concrete. A weak objective is 'to study mobile applications'. A strong objective is 'to design and build a working prototype that lets students search for, book and cancel

study rooms'. Each objective should name an action and a deliverable: design, build, test, evaluate. Aim for four to six objectives, in the order they will be carried out, and make sure that together they cover the whole project. If an objective cannot be checked at the end — if you could not say whether it was done — rewrite it until it can be.

3.3 Common proposal mistakes

The most common mistake is promising too much: a proposal that lists ten features, three platforms and a new algorithm will not be believed. The second mistake is a proposal with no method: it says what will be built but not how, with what tools, or how it will be tested. The third is a background section that is copied from the internet without understanding; supervisors check, and a copied proposal is the fastest route to rejection.

Worked Example: A weak objective and a strong one

Weak: 'To learn about databases and improve my programming.' This is a personal goal, not a project objective; it cannot be checked. Strong: 'To build a database-backed booking system with three functions — search, book and cancel — and to test it with ten students, recording the time each booking takes.' The strong version names the deliverable, the functions and the test. At the end of the project, anyone can check whether it was done.

[ ] The proposal states the problem, the aim and the objectives.

[ ] Objectives are concrete, checkable and in the order they will be done.

[ ] The background shows what exists and what is missing.

[ ] The method names the tools, the data and the testing.

[ ] A realistic timeline is included, with time for writing and testing.

Chapter 4: Planning and Timeline: Work Breakdown and Gantt-Style Planning

A final year project is a long job with a fixed deadline, and the students who finish it are rarely the most talented. They are the ones who planned in weeks, not wishes. Planning has two parts: breaking the work into small pieces, and putting those pieces on a calendar. The work breakdown turns a frightening whole — 'build a system' — into a list of tasks that can each be done in a few days. The timeline turns that list into dates, so that the student always knows whether the project is on schedule, and how much time is left for the parts that matter most: testing and writing.

4.1 The work breakdown structure

Break the project into phases, then into tasks. The phases are usually: background study, design, implementation, testing, evaluation, and writing the report. Under each phase, list the tasks. Under implementation, for example, the tasks might be: set up the tools, build the database, build the booking function, build the cancellation

function, and connect the parts together. Each task should be small enough to finish in a few days, and each should have a clear result: a working function, a tested page, a written section. If a task cannot be finished in a week, break it into smaller tasks. A plan made of small tasks is easy to track; a plan made of large tasks is easy to hide behind.

4.2 Gantt-style planning

A Gantt chart is a simple bar chart: tasks down the left, weeks across the top, and a bar for each task showing when it starts and ends. It is the standard way to show a project plan, and most departments expect one in the proposal. Draw it honestly. Put testing and writing in the plan as real tasks with real weeks, not as a single bar at the end. Build in slack — at least two or three weeks of spare time across the project — because something will go wrong: a tool will fail, a user will be unavailable, a function will take twice as long as expected. A plan without slack is a plan that fails at the first problem. Update the chart every week: mark tasks done, move the bars that slipped, and tell your supervisor early when the plan changes.

4.3 Weekly rhythm and milestones

Set milestones: dates by which a visible piece of work will exist. A milestone is not 'work on the database'; it is 'the database is built and holds test data'. Aim for a milestone every two or three weeks, and show each one to your supervisor. Between milestones, keep a steady weekly rhythm: a fixed number of hours each week, with a short note at the end of each week saying what was done and what comes next. Students who work in bursts — nothing for a month, then a frantic week — produce worse work and more stress than students who work steadily.

Phase	Typical share of tim	Milestone
Background study	10-15%	Background chapter drafted
Design	10-15%	Design diagrams approved
Implementation	30-35%	Working version demonstrated
Testing and evaluation	15-20%	Test results recorded
Report writing	20-25%	Full draft to supervisor
Revision and submission	5-10%	Final report submitted

[ ] The work is broken into tasks, each finishable in a few days.

[ ] A Gantt-style chart shows tasks against weeks.

[ ] Testing and writing have real weeks in the plan, not a bar at the end.

[ ] At least two weeks of slack are built into the plan.

[ ] Milestones are set every two or three weeks and shown to the supervisor.

Chapter 5: Literature and Background Study

Before building anything, a good project studies what already exists. The background study has two purposes: to learn from earlier work so that the project does not repeat known mistakes, and to show the examiner that the project is grounded in the subject rather than invented from nothing. It covers two kinds of material: published literature — books, journal articles, and reputable technical sources — and existing systems, products or solutions that do the same or a similar job. Both are reported in the same chapter, and both are judged by the same standard: does the student understand the material, and can they say what is missing?

5.1 Searching and selecting sources

Start with your university library and its databases, then use reputable technical sources for your subject. Keep a record of every search: the terms used, the database, and the number of results. Read the most relevant sources in full, and skim the rest. For each source, note the problem it addressed, the method it used, and its main finding. Be selective: ten sources understood deeply are worth more than fifty listed without comment.

5.2 Reviewing existing systems

For a development project, the most useful background material is often the existing systems that do a similar job. Find two or three of them — real products, not imaginary ones — and study them as a user. What do they do well? What is missing? What would your users need that these systems do not provide? Present the comparison in a table, with the systems as columns and the features as rows, so the gap your project fills is visible at a glance. This comparison is one of the strongest parts of a project report, because it shows that the project solves a real gap rather than duplicating something that already exists.

5.3 From background to requirements

The background chapter should end with the requirements for your own project: the list of things the system, study or design must do. Each requirement should be traceable to something in the background — a feature users need, a weakness in an existing system, a gap in the literature. Number the requirements, and use the same numbers in the design chapter and in the testing chapter, so that the examiner can follow a single requirement from the background, through the design, to the test that proves it works. This traceability is one of the clearest signs of a well-run project.

Existing system	Strengths	Weaknesses	What my project ad

General booking apps	Easy to use; reminders	Not made for study rooms	Room maps and time slots
Paper sign-up sheets	Simple; no technology	No record; rooms double-booked	Live availability and history
University portal	Official; trusted	Slow; no booking by room	Fast booking by room and time

[ ] At least ten relevant sources have been read and understood.

[ ] Two or three existing systems have been studied as a user.

[ ] A comparison table shows what exists and what is missing.

[ ] Requirements are numbered and traceable to the background.

[ ] Every source is recorded with full details for the reference list.

Chapter 6: Design and Methodology

Design is the bridge between the requirements and the finished product. It answers the question 'how will this work?' before a single line of code is written or a single part is ordered. A project without a design chapter is a project that was built by trial and error, and examiners can tell: the product may work, but the report cannot explain why it is built the way it is, or what alternatives were considered and rejected. The design chapter should be detailed enough that a competent reader could understand the structure of the solution, and it should be written before the building starts, so that the building follows the design rather than the other way round.

6.1 The overall design

Begin with the overall structure: the main parts of the system and how they fit together. For a software project, this is usually a diagram showing the parts — the user interface, the application logic, the database — and the flow of information between them. For a hardware project, it is a block diagram of the components. For a research project, it is the design of the study: the participants, the instruments, the procedure and the analysis. Describe each part in a short paragraph, and explain the choice: why this structure and not another, what the alternatives were, and what the trade-offs are. A design choice without a reason looks arbitrary; a design choice with a reason looks like engineering.

6.2 Detailed design: diagrams and descriptions

After the overall structure, describe the important details. For software, this means the database tables and their fields, the main screens and what each one does, and

the key algorithms or processes in plain language. For hardware, it means the circuit or mechanical layout and the choice of components. For a study, it means the questionnaire or interview schedule and the plan for analysis. Use diagrams wherever a diagram is clearer than words — a database diagram, a flow chart, a screen sketch — and number and label every diagram. Put long details, such as the full list of database fields, in an appendix, and refer to it from the chapter.

6.3 Methodology: how the project will be carried out

The methodology section explains the approach to the work itself. Name the development approach — for example, building in stages with a working version at the end of each stage, and testing each stage before moving on — and explain why it fits this project. State the tools and technologies chosen, with a reason for each: the programming language, the database, the hardware, the testing tools. State how data will be collected, if the project involves users or measurements, and how ethical issues will be handled: consent from participants, confidentiality, and secure storage of data. A methodology that is written honestly, including its limits, earns more marks than one that claims a method the student did not actually follow.

- [ ] An overall diagram shows the parts of the solution and how they connect.
- [ ] Each design choice is justified, with alternatives considered.
- [ ] Database, screens or components are described in enough detail to understand.
- [ ] Tools and technologies are named with a reason for each.
- [ ] Ethical issues, where the project involves people, are addressed.

Chapter 7: Implementation and Development

Implementation is the building phase: writing the code, assembling the device, running the study, or producing the design. It is the longest phase of most projects, and it is also the phase where the plan is most often lost. The discipline of implementation is to build in stages, test each stage, and keep a record of what was done. A student who builds the whole system before testing anything will discover the faults all at once, at the end, when there is no time to fix them. A student who builds in stages discovers faults one at a time, while there is still time.

7.1 Building in stages

Divide the implementation into stages that match the design: first the foundation — the database, the basic structure, the core functions — then the features, one at a time, and finally the finishing touches. At the end of each stage, produce a working version that can be shown to your supervisor, even if it is incomplete. A working version is worth more than a promise: it proves progress, it gives the supervisor something concrete to comment on, and it protects the student if something goes wrong later, because there is always a working version to fall back on. Keep a dated copy of each working version, and never overwrite the only copy of a version that works.

7.2 Keeping a development log

Keep a simple log of the implementation: the date, what was built, what problems appeared, and how they were solved. The log serves three purposes. It is evidence of the work done, it helps the student remember decisions that will need to be explained in the report, and it provides the material for the implementation chapter. The most useful entries are the problems and their solutions: a tool that would not install, a function that failed in a particular case, a design that had to be changed. Examiners value a report that describes real problems and real solutions, because it shows that the student did the work and understood it. A report that claims everything worked first time is not believed.

7.3 When the design must change

Designs change during implementation. A feature turns out to be harder than expected, a tool does not do what was promised, a user asks for something different. Changing the design is normal, and it is not a failure — but the change must be recorded, with the reason, and the report must describe the design as it was finally built, not as it was first planned. If a feature had to be dropped, say so plainly, explain why, and state what would be needed to add it in future work. Examiners respect a student who makes a sensible cut and explains it; they do not respect a student who pretends the cut never happened and leaves the report describing a feature that does not exist.

7.4 Version control and backups

Use version control for any software project, and keep backups of everything else. Version control records every change and lets the student return to an earlier working version. Even a simple system — dated folders, with a note of what changed — is far better than a single folder that is overwritten daily. Back up to a second location at least once a week, and before any major change.

[ ] The system is built in stages, each ending in a working version.

[ ] A development log records dates, work done, problems and solutions.

[ ] Design changes are recorded with their reasons.

[ ] Version control or dated copies protect every working version.

[ ] Backups are kept in a second location and updated weekly.

Chapter 8: Testing and Evaluation

Testing answers the question 'does it work, and does it do what it was supposed to do?' Evaluation answers the harder question: 'is it any good?' Both belong in the project, and both must be planned before the testing starts, not invented afterwards. A test plan lists what will be tested, how it will be tested, what the expected result is, and what actually happened. An evaluation goes further: it asks real users to try the system, or compares the results with the requirements, and reports what was found — including what failed. A project that reports only success has not been tested; a project that reports its failures honestly has been tested properly.

8.1 The test plan

Write the test plan from the requirements: every requirement should have at least one test. For each test, record the test number, the requirement it checks, the steps to carry out, the expected result, and the actual result. Test the normal cases first, then the unusual ones: what happens when the input is empty, when two users book the same room at the same time, when the network fails? Record every test, including the ones that failed, and record what was done to fix each failure. The test table in the report is the evidence that the system was tested, and examiners read it carefully — a table in which every test passed on the first attempt is a sign that the testing was not thorough.

8.2 User evaluation

Where the project has users, ask them to try it. Five to ten users are usually enough for an undergraduate project. Give each user a short task — for example, 'book a study room for tomorrow at 10 a.m.' — and observe, without helping, how long it takes and where they get stuck. Afterwards, ask a few short questions: what was easy, what was confusing, what would they change? Record the time taken, the errors made, and the comments, and report them plainly.

8.3 Evaluating against the requirements

The final step is to go through the numbered requirements from Chapter 5 and state, for each one, whether it was fully met, partly met, or not met, with the evidence. This requirements check is the most honest page in the report, and it is often the page examiners trust most. A project that meets eight of ten requirements, and says so, earns more credit than a project that claims to meet all ten while the demonstration shows otherwise. Where a requirement was not met, explain why, and say what would be needed to meet it in future work.

Test	Requirement checked	Expected result	Actual result
T1	Search for a free room	List of free rooms shown	Pass
T2	Book a room	Booking saved; confirmation shown	Pass
T3	Book the same room twice	Second booking refused	Fail - fixed, retest pass
T4	Cancel a booking	Room freed; booking removed	Pass

T5	Empty search	Clear message, no crash	Pass

[ ] A test plan covers every requirement, with expected and actual results.

[ ] Unusual and error cases have been tested, not only normal ones.

[ ] Failures are recorded with the fix and the retest.

[ ] Real users have tried the system and their results are reported.

[ ] Each requirement is checked as fully met, partly met or not met.

Chapter 9: Results

The results chapter presents what the project produced and what the tests found — nothing more. It does not interpret the findings, compare them with other work, or explain why they happened. That is the discussion chapter's job. The results chapter's job is to give the reader a clear, checkable picture: what was built, what it does, and how it performed when tested. The most common fault in this chapter is mixing results with opinion, so that the reader cannot tell where the evidence ends and the author's judgement begins.

9.1 Presenting the product

Begin by describing what was delivered, with screenshots or photographs of the main screens or the finished device. Show the most important parts first: the main screen, the key function, the result a user sees. Number and caption every figure, and refer to it in the text. Keep screenshots large enough to read, and do not include every screen — choose the ones that show the product working. If the product is a study rather than a built system, present the data instead: the measurements, the counts, the survey results, in tables and charts that a reader can follow.

9.2 Presenting the test and evaluation results

Present the test results in the order of the test plan, using the test table from Chapter 8 as the backbone. Summarise the overall result first — how many tests passed, how many failed and were fixed — then draw attention to the results that matter most. Present user evaluation results with numbers and with short quotations: how long the tasks took, how many errors occurred, and what users said.

9.3 Reporting negative results honestly

Report what did not work. A feature that failed, a test that could not be run, a requirement that was not met — these are results, and they belong in this chapter. Hiding them is a serious fault: examiners will find them during the demonstration, and a report that hid them loses trust in everything else it claims. State each negative result plainly, with the evidence, and leave the explanation and the remedy for the discussion chapter. Honest reporting of failure is one of the clearest signs that a student understands what a project report is for.

[ ] The product is shown with numbered, captioned figures.

[ ] Test results are presented in the order of the test plan.

[ ] User evaluation results give numbers and quotations.

[ ] No data appear in both a table and a chart.

[ ] Negative results are reported, not hidden.

Chapter 10: Discussion

The discussion is where the report earns its marks. The results chapter showed what happened; the discussion argues what it means. A strong discussion does four things: it answers the project's aim directly, it explains the results, it compares them with the background study, and it admits the limitations honestly. Students often write the discussion as a second results chapter, repeating the numbers. Resist that. The reader has already seen the numbers; now they need your judgement, and your judgement must be tied to the evidence.

10.1 Answer the aim first

Begin the discussion by answering the aim in two or three sentences, referring back to the key evidence. Do not make the reader wait until the end of the chapter. A useful opening is: 'The project aimed to reduce the time students spend finding a free study room. The evidence suggests that it does: the average booking time fell from 3 minutes with the paper system to 1 minute 40 seconds with the application, and nine of ten test users completed a booking without help.' Everything after that sentence supports, qualifies or explains it. If the aim was not fully met, say so in the same place, and say which part was met and which was not.

10.2 Explaining the results

For each main result, offer the most likely explanation, and say what evidence supports it. If users booked faster, is it because the search was quicker, because the interface was clearer, or because the test users were already familiar with similar applications? Consider more than one explanation, and say which the evidence favours. Also explain the unexpected results: the student who could not find the booking button, the test that failed, the requirement that was only partly met.

10.3 Comparing with the background

Return to the background study and say whether your results agree with what was known, and whether your solution improves on the existing systems you reviewed. If an existing system was slow or hard to use, and yours is faster or easier, say so, with the numbers. If your results disagree with what the literature suggested, say so too, and offer a reason: a different group of users, a different setting, a smaller sample. Do not ignore the sources that contradict your findings. Examiners will have read your background chapter, and a discussion that confronts contrary evidence is far stronger than one that pretends it does not exist.

10.4 Limitations

State the limitations plainly: the small number of test users, the short testing period, the single site, the features that were not built. Then say what each limitation means for the findings: which conclusions it weakens and which it leaves untouched. A limitation is not an apology. Every project has limits, and naming them shows that you understand the reach of your own evidence. What damages a report is not having limitations, but pretending not to have them.

[ ] The aim is answered directly, early in the chapter.

[ ] Each main result is explained, with the evidence for the explanation.

[ ] Unexpected results are explained, not ignored.

[ ] Findings are compared with the background study and existing systems.

[ ] Limitations are stated with their consequences for the findings.

Chapter 11: Conclusion and Future Work

The conclusion is short, and it must be. By this point the argument has been made; the conclusion states what was achieved and what follows from it. Students often write a conclusion that is either a long summary repeating the whole report, or a set of grand claims the evidence cannot support. Both are mistakes. Aim for a chapter the examiner can read in five minutes and remember afterwards: the aim, the answer, the contribution, and what should happen next.

11.1 Restating the aim and the answer

Open by restating the aim and giving the answer in two or three sentences. This is the project in miniature, and it should be the clearest passage in the whole report. Then summarise the contribution: what was built or found that did not exist before, and for whom it matters. Keep the summary to the main achievements only; the detail belongs in the earlier chapters. Do not introduce new results or new claims in the conclusion. If a claim is not supported by a chapter earlier in the report, it does not belong here.

11.2 Future work

Future work follows from the limitations and from the discussion. List the improvements that would come next, in order of importance: the feature that was cut for lack of time, the test that needs more users, the setting in which the system should be tried next. For each item, say briefly what would be involved and what it would show. Be realistic: future work is a plan, not a wish list, and it should follow from what this project actually did. A future-work section that is honest about the project's limits is more convincing than one that promises a revolution.

11.3 A final note on honesty

The conclusion is the last thing the examiner reads before forming a judgement, and its tone matters. State the achievement plainly, without exaggeration, and state the limits plainly too. A report that says 'the system reduced booking time in a ten-user test at one campus, and should next be tested more widely' states exactly what the

evidence supports — and that earns trust, which is what the whole report is built on.

[ ] The aim is restated and answered in two or three sentences.

[ ] The contribution is summarised in the main achievements only.

[ ] No new results or claims appear in the conclusion.

[ ] Future work follows from the limitations and is listed in order of importance.

[ ] Claims are stated no more strongly than the evidence supports.

Chapter 12: Report Structure and Formatting

A project report follows a standard structure that examiners expect, and following it makes the report easier to read and easier to mark. The structure below is used in most universities, but always check your department's regulations first: the required order, the word count, the page size and the referencing style are set by your university, not by this guide. The table shows the parts of a typical report of about 8,000 to 12,000 words; shorter reports compress the same parts without changing the order.

Part	Purpose	Typical share
Title page	Title, author, course, date	1 page
Abstract	Summary of the whole report	200-300 words
Contents	List of sections and page numbers	1 page
1. Introduction	Problem, aim, objectives, outline	8-10%
2. Background	Literature and existing systems	12-15%
3. Design	Overall and detailed design	12-15%
4. Implementation	How the solution was built	15-20%
5. Testing and Evaluation	Test plan, results, user evaluation	12-15%
6. Results	What was produced and found	10-12%
7. Discussion	What the results mean	8-10%

8. Conclusion	Answer, contribution, future work	4-6%
References	Sources cited	As needed
Appendices	Code extracts, logs, test data	As needed

12.1 Formatting rules that matter

Follow your university's template exactly: font, font size, line spacing, margins, heading styles, page numbering and the position of tables and figures. If no template is provided, use a readable font at 11 or 12 points, 1.5 line spacing, and margins of at least 2.5 centimetres. Number chapters and sections consistently, start each chapter on a new page, and keep headings with at least two lines of text beneath them.

12.2 Referencing and academic honesty

Choose the referencing style your department requires and use it without exception. Record the full details of every source as you read it, and use a reference manager to keep the list consistent. Check three things before submission: every citation in the text appears in the reference list; every entry in the reference list is cited in the text; and every reference is complete and correctly punctuated. Present your own work as your own and other people's work as theirs. Code or text taken from elsewhere must be credited and explained, and your department's rules on the use of libraries, frameworks and generated material must be followed. Plagiarism — presenting someone else's work as your own — is the most serious fault in academic work, and it can lead to failure of the project.

12.3 Proofreading

Proofread in stages, not in one sitting. First, read for structure: does each chapter do its job, and does the argument flow? Second, read for sense: does each paragraph make one point? Third, read for language: spelling, grammar, punctuation, consistency of terms and numbers. Read the report aloud — the ear catches errors the eye misses. Budget at least a week for proofreading and correction, and do not leave it to the last night.

- [ ] The report follows the standard structure in the required order.
- [ ] The university template and referencing style are followed throughout.
- [ ] Every table and figure is numbered, captioned and referred to in the text.
- [ ] Every citation has a matching reference, and vice versa.
- [ ] The report has been read aloud at least once.

Chapter 13: The Presentation, Demonstration and Viva

Most final year projects end with a presentation, a demonstration and a viva — an oral examination of the work. The demonstration shows that the product works. The presentation explains what was done and what was found. The viva tests whether the student understands the work and can defend it. Students often treat these as an afterthought, prepared in the last week. They are not: a good demonstration and a confident viva can lift a project by a whole grade, and a failed demonstration can sink a good report.

13.1 The demonstration

Prepare the demonstration as carefully as the report. Choose a short sequence of actions that shows the product working — the main functions, in the order a user would use them — and rehearse it until it runs smoothly. Prepare test data in advance, and make sure the system works on the machine that will be used for the demonstration, not only on your own laptop. Have a backup: a video recording of the system working, or screenshots, in case the live demonstration fails.

13.2 The presentation

A presentation of ten to twenty minutes is normal. Structure it like the report in miniature: the problem, the aim, the design, the demonstration, the results, and the conclusion. Use few slides, large text, and one clear visual per point. Do not read the slides aloud; the examiners have the report. Rehearse with a timer, and rehearse standing up, because that is how you will deliver it. Time the demonstration inside the rehearsal, so that the whole presentation fits its slot. End with the conclusion and the future work, and invite questions.

13.3 The viva: what examiners ask

Examiners ask the same kinds of questions in almost every viva. Why did you choose this topic? What is original about your project? Why did you choose this design, this tool, this method, rather than another? How do you explain this unexpected result? What are the limitations? What would you do differently if you started again? Prepare a two-minute answer to each of these before the day, and practise saying them aloud. On the day, bring a copy of the report and a notebook. Listen to each question in full before answering, and if you do not understand a question, ask the examiner to rephrase it. It is acceptable to say 'I need a moment to think' or 'I do not know, but the evidence suggests...'. Never invent an answer.

[ ] The demonstration sequence is chosen, rehearsed and timed.

[ ] Test data are prepared and the system works on the demonstration machine.

[ ] A backup recording or screenshots exist in case the live demo fails.

[ ] The presentation fits its time slot, including the demonstration.

[ ] Two-minute answers are prepared for the standard viva questions.

Chapter 14: Working with Your Supervisor

The supervisor is the most important resource in the project, and the relationship is a professional one: the student does the work, and the supervisor advises, questions and guides. Supervisors are busy, and their time is limited, so the student must use it well. The students who get the most from supervision are the ones who arrive with something to show — a draft, a working version, a specific question — and who act on the advice they are given. The students who get the least are the ones who arrive with nothing and ask 'what should I do next?'

14.1 Meetings that work

Agree a regular meeting pattern at the start — for example, every two weeks — and keep to it, even when there is little to report. Before each meeting, send a short note: what was done since the last meeting, what problems appeared, and what you want to discuss. Bring the work itself: a draft chapter, a working version, a test result. Take notes during the meeting, and send a short summary afterwards, so that both sides agree on what was decided. If a meeting must be missed, say so in advance. A supervisor who is kept informed is a supervisor who can help; a supervisor who hears nothing for two months cannot.

14.2 Using feedback well

Feedback is information, not criticism. When your supervisor marks a draft, read the comments twice: once to understand what is wrong, and once to decide what to change. If a comment is unclear, ask. If you disagree, say so, with a reason — supervisors respect a student who can defend a decision, and they will change their mind when the reason is good. Keep a list of the feedback received and what was done about it, and show the revised draft at the next meeting. A student who returns with the same errors uncorrected, meeting after meeting, will find the supervisor's patience wearing thin.

14.3 When things go wrong

Projects go wrong: a tool fails, a user withdraws, a key feature cannot be built, a deadline slips. Tell your supervisor early. A problem reported in week six can be solved by narrowing the scope or changing the design. The same problem reported in week twenty cannot. Be honest about the cause, propose a solution, and ask for advice — 'the database tool is not working; I propose to switch to the simpler tool we discussed, which will cost two weeks; is that acceptable?' is a message a supervisor can act on. Hiding a problem until it is too late is the one mistake supervisors find hardest to forgive.

[ ] A regular meeting pattern is agreed and kept.

[ ] A short progress note is sent before each meeting.

[ ] Feedback is recorded and acted on before the next meeting.

[ ] Problems are reported early, with a proposed solution.

[ ] Decisions from each meeting are summarised and agreed.

Chapter 15: Common Mistakes and a Checklist

The mistakes below appear in final year projects year after year, in every subject. They are not mistakes of intelligence; they are mistakes of planning, and every one of them is avoidable. Read this chapter before you choose a topic, not only before you submit: by submission time, the project mistakes are already made.

15.1 The ten most common mistakes

- Choosing a topic that is far too large, and running out of time with nothing finished.
- Writing the proposal with promises that cannot be kept, and spending the year failing to deliver them.
- No plan, or a plan with no slack, so the first problem destroys the schedule.
- Building the whole system before testing anything, and discovering all the faults at the end.
- No design chapter, or a design written after the building, so the report cannot explain the choices.
- A background chapter that lists sources without comparing them or finding the gap.
- Test results that are invented, incomplete, or show only success.
- A report that describes a product that does not work, or a feature that does not exist.
- Leaving the writing to the last weeks, and submitting an unproofread draft.
- Hiding the limitations and the failures, instead of stating them honestly.

15.2 The final checklist

Work through this checklist with the finished report and the working product side by side. Tick each item only when you have checked it, not from memory.

[ ] The aim is stated in the introduction and answered in the conclusion.

[ ] Objectives are concrete and each one is checked as done or not done.

[ ] The design chapter explains the choices, with alternatives considered.

[ ] Every requirement has a test, and the test results are shown.

[ ] The product works and has been demonstrated to the supervisor.

[ ] User evaluation, where the project has users, is reported honestly.

[ ] Limitations are stated with their consequences for the findings.

[ ] Every table and figure is numbered, captioned and referred to in the text.

[ ] Every citation appears in the reference list, and every reference is cited.

[ ] No feature is described that does not exist in the product.

[ ] Page numbers, headings and the table of contents match.

[ ] The report has been proofread, and the file is backed up in a second location.

Chapter 16: Conclusion

The final year project is the most demanding part of most degrees, and also the most rewarding. It is demanding because it cannot be done by following instructions: the student must choose the problem, plan the work, solve the problems that appear, and write about it all clearly. It is rewarding because it produces something real — a working system, a study, a design — and because it teaches skills that no examination can teach: managing time, making decisions with incomplete information, recovering from failure, and explaining complex work to other people.

The habits in this guide are simple, and they are the whole method. Choose a topic that is real, specific and finishable. Write a proposal that promises only what you can deliver. Plan in weeks, with slack and milestones. Study what already exists, and design before you build. Build in stages, test each stage, and keep a log of problems and solutions. Test with real users, report the failures honestly, and discuss what the results mean. Work with your supervisor, report problems early, and write the report as you go, not at the end. Do that, and the project will be what a final year project should be: finished, tested, honestly reported, and yours.

Suggested Reading

The following are general kinds of works that students commonly consult for final year projects. Check your university library for the latest editions, and follow your department's guidance on referencing style.

- Standard research methods textbooks: general methodology texts used in most universities cover research design, data collection, analysis and report writing for quantitative, qualitative and mixed-methods studies.
- Software engineering and project management guides: widely used works on building software in stages, testing, and managing a project explain the methods this guide summarises, with examples from real projects.
- Academic writing guides: standard guides to writing reports, theses and dissertations explain structure, argument and style, with examples of good practice at undergraduate and postgraduate level.
- Style manuals: the official manuals of the major referencing styles (APA, Harvard, Chicago, IEEE) are the authoritative source for citation and formatting rules — consult the edition your department requires.
- Your university's final year project handbook and regulations: the first and final authority on structure, word count, marking criteria, deadlines and submission requirements.
- Your supervisor's recommended reading list for your subject: the most relevant source of all, because it is chosen for your specific topic and method.